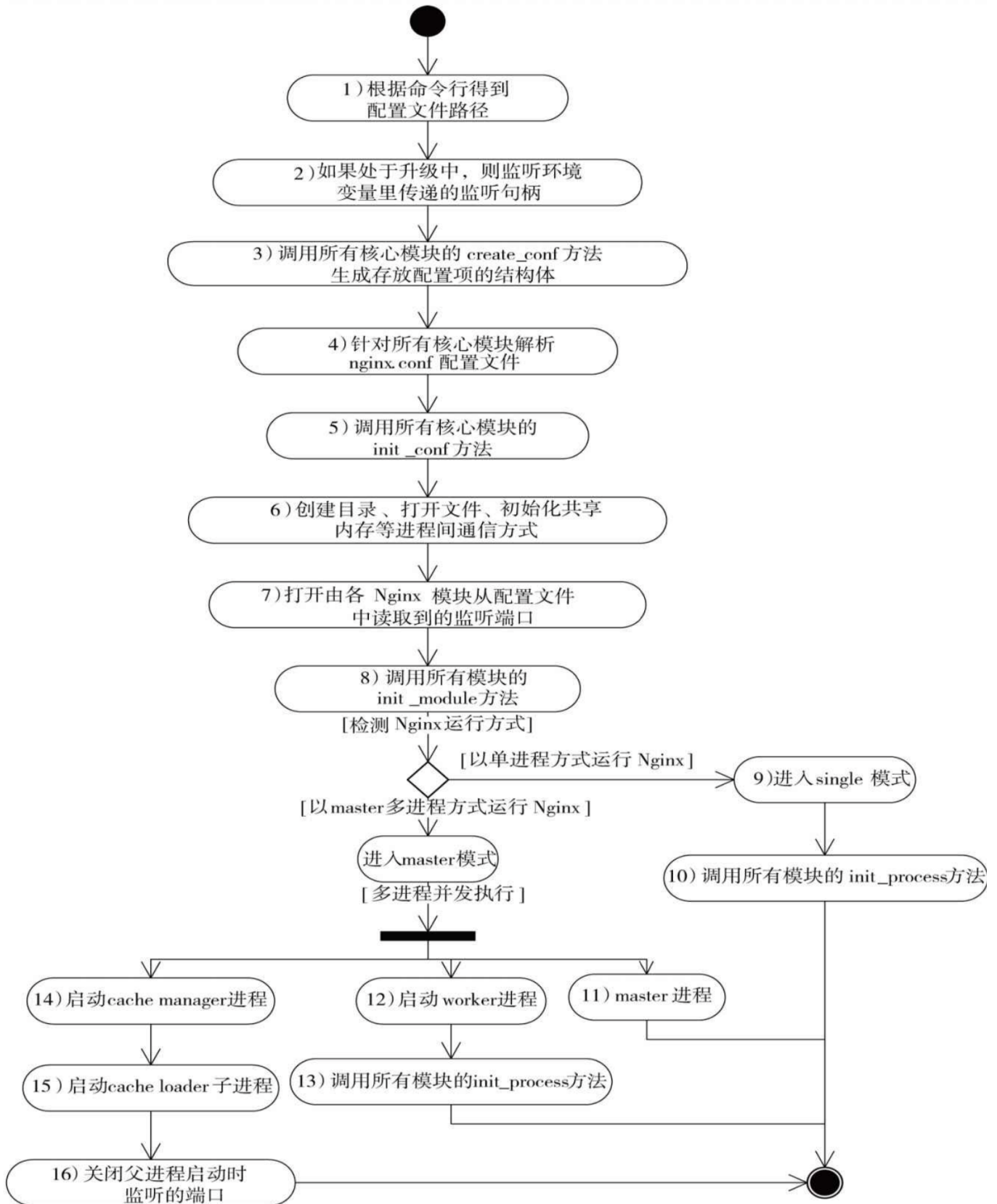


事件驱动模型

从模块构造说起

core模块 (NGX_CORE_MODULE)	conf模块 (NGX_CONF_MODULE)	event模块 (NGX_EVENT_MODULE)	http模块 (NGX_HTTP_MODULE)	mail模块 (NGX_MAIL_MODULE)
ngx_core_module ngx_errlog_module ngx_regex_module ngx_events_module ngx_http_module ngx_mail_module	ngx_conf_module	ngx_event_core_module ngx_devpoll_module ngx_epoll_module ngx_eventport_module ngx_kqueue_module ngx_poll_module ngx_select_module	ngx_http_core_module ngx_http_log_module ngx_http_upstream_module ngx_http_static_module (http模块较多，暂不一一列举)	ngx_mail_core_module

总体启动流程图



模块数据的存储

```
static ngx_command_t  ngx_events_commands[] = {

    { ngx_string("events"),
      NGX_MAIN_CONF|NGX_CONF_BLOCK|NGX_CONF_NOARGS,
      ngx_events_block,
      0,
      0,
      NULL },

    ngx_null_command

};

static ngx_core_module_t  ngx_events_module_ctx = {
    ngx_string("events"),
    NULL,
    ngx_event_init_conf
};

ngx_module_t  ngx_events_module = {
    NGX_MODULE_V1,
    &ngx_events_module_ctx,          /* module context */
    ngx_events_commands,             /* module directives */
    NGX_CORE_MODULE,                 /* module type */
    NULL,                            /* init master */
    NULL,                            /* init module */
    NULL,                            /* init process */
    NULL,                            /* init thread */
    NULL,                            /* exit thread */
    NULL,                            /* exit process */
    NULL,                            /* exit master */
    NGX_MODULE_V1_PADDING
};
```

ngx_events_module 模块配置初始化

```
static char *
ngx_event_init_conf(ngx_cycle_t *cycle, void *conf)
{
    if (ngx_get_conf(cycle->conf_ctx, ngx_events_module) == NULL) {
        ngx_log_error(NGX_LOG_EMERG, cycle->log, 0,
            "no \"events\" section in configuration");
        return NGX_CONF_ERROR;
    }
}
```

```
}

return NGX_CONF_OK;

}
```

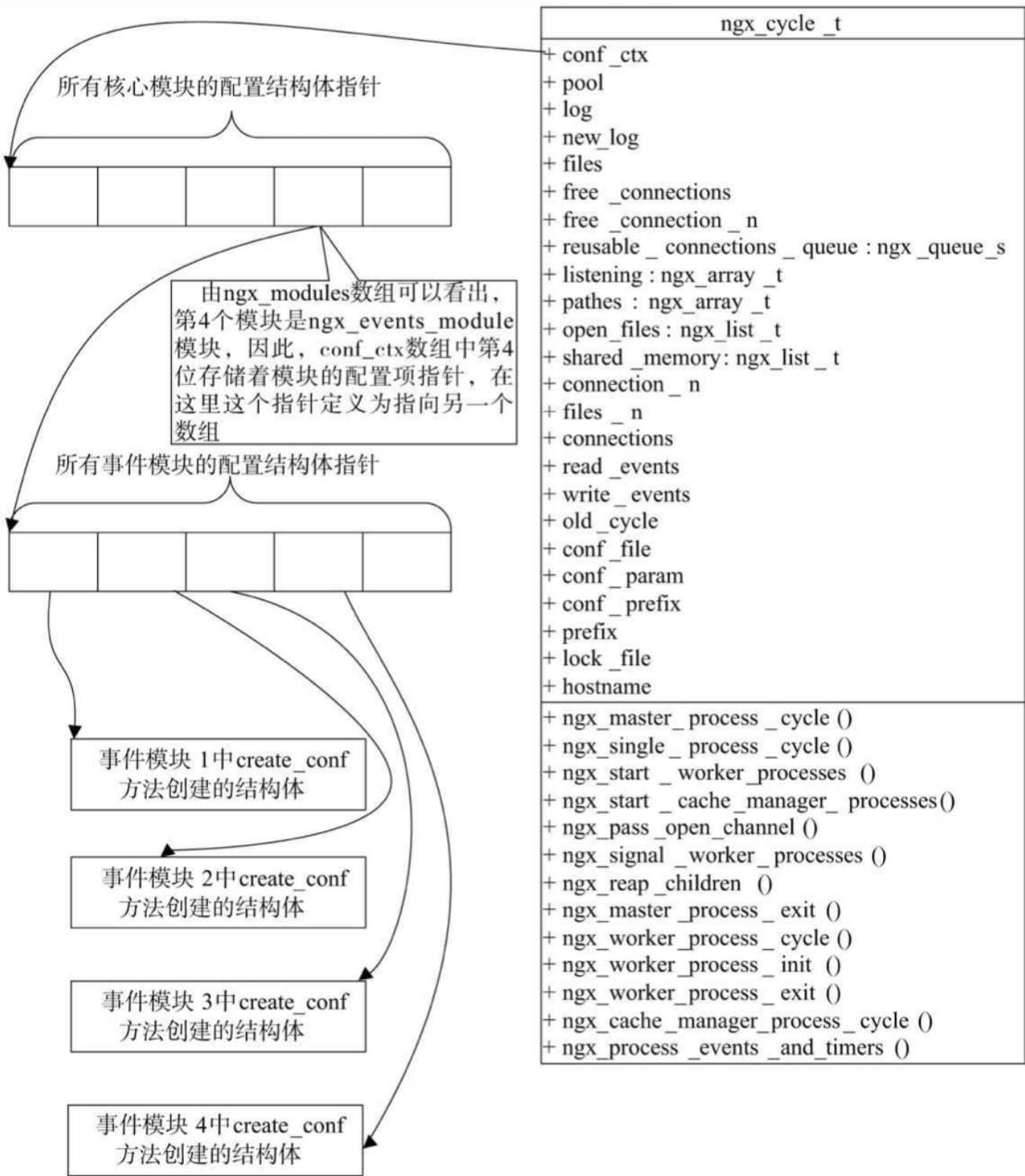


图9-2 所有事件模块配置项结构体的指针是如何管理的

核心模块配置加载

可以看到ngx_events_module 模块仅仅实现了ngx_event_init_conf接口进行配置解析，最终调用接口为ngx_events_block接口。

ngx_events_block调用流程

```
/**
 * 模块解析
 * 事件模块配置如下:
 * events {
     worker_connections 1024;
 }
    光使用核心配置的方式，只能解析到 events 这一层。
    如果继续往{}中的内容解析，就得重新调用ngx_conf_parse进行解析
 */

static char *
ngx_events_block(ngx_conf_t *cf, ngx_command_t *cmd, void *conf) {
    char *rv;
    void **ctx;
    ngx_uint_t i;
    ngx_conf_t pcf;
    ngx_event_module_t *m;

    if (*(void **) conf) {
        return "is duplicate";
    }

    /* count the number of the event modules and set up their indices */

    ngx_event_max_module = ngx_count_modules(cf->cycle, NGX_EVENT_MODULE);

    /* 分配内存空间 */
    ctx = ngx_palloc(cf->pool, sizeof(void *));
    if (ctx == NULL) {
        return NGX_CONF_ERROR ;
    }

    *ctx = ngx_palloc(cf->pool, ngx_event_max_module * sizeof(void *));
    if (*ctx == NULL) {
        return NGX_CONF_ERROR ;
    }

    *(void **) conf = ctx;

    /* 模块初始化，如果是NGX_EVENT_MODULE，则调用模块的create_conf方法 */
    for (i = 0; cf->cycle->modules[i]; i++) {
        if (cf->cycle->modules[i]->type != NGX_EVENT_MODULE) {
            continue;
        }
    }
}
```

```
    m = cf->cycle->modules[i]->ctx;

    if (m->create_conf) {
        (*ctx)[cf->cycle->modules[i]->ctx_index] = m->create_conf(
            cf->cycle);
        if ((*ctx)[cf->cycle->modules[i]->ctx_index] == NULL) {
            return NGX_CONF_ERROR ;
        }
    }
}

/* event*/
pcf = *cf;
cf->ctx = ctx;
cf->module_type = NGX_EVENT_MODULE;
cf->cmd_type = NGX_EVENT_CONF;

/* 调用配置解析，这次解析的是 块中的内容，非文件内容 */
rv = ngx_conf_parse(cf, NULL);

*cf = pcf;

if (rv != NGX_CONF_OK) {
    return rv;
}

/* 初始化 模块的init_conf 方法*/
for (i = 0; cf->cycle->modules[i]; i++) {
    if (cf->cycle->modules[i]->type != NGX_EVENT_MODULE) {
        continue;
    }

    m = cf->cycle->modules[i]->ctx;

    if (m->init_conf) {
        rv = m->init_conf(cf->cycle,
            (*ctx)[cf->cycle->modules[i]->ctx_index]);
        if (rv != NGX_CONF_OK) {
            return rv;
        }
    }
}

return NGX_CONF_OK;
}
```

nginx_event_core_module 模块

```

static ngx_event_module_t ngx_event_core_module_ctx = {
    &event_core_name,
    ngx_event_core_create_conf,          /* create configuration */
    ngx_event_core_init_conf,           /* init configuration */

    { NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL }
};

ngx_module_t ngx_event_core_module = {
    NGX_MODULE_V1,
    &ngx_event_core_module_ctx,          /* module context */
    ngx_event_core_commands,             /* module directives */
    NGX_EVENT_MODULE,                   /* module type */
    NULL,                                /* init master */
    ngx_event_module_init,               /* init module */
    ngx_event_process_init,              /* init process */
    NULL,                                /* init thread */
    NULL,                                /* exit thread */
    NULL,                                /* exit process */
    NULL,                                /* exit master */
    NGX_MODULE_V1_PADDING
};

```

ngx_core_commands 解析命令

```

static ngx_command_t ngx_core_commands[] = {
    { ngx_string("daemon"),
      NGX_MAIN_CONF | NGX_DIRECT_CONF | NGX_CONF_FLAG,
      ngx_conf_set_flag_slot,
      0,
      offsetof(ngx_core_conf_t, daemon),
      NULL },
    { ngx_string("master_process"),
      NGX_MAIN_CONF | NGX_DIRECT_CONF | NGX_CONF_FLAG,
      ngx_conf_set_flag_slot,
      0,
      offsetof(ngx_core_conf_t, master),
      NULL },
    { ngx_string("timer_resolution"),
      NGX_MAIN_CONF | NGX_DIRECT_CONF | NGX_CONF_TAKE1,
      ngx_conf_set_msec_slot,
      0,
      offsetof(ngx_core_conf_t, timer_resolution),
      NULL },
};

```

```

{ ngx_string("pid"),
NGX_MAIN_CONF | NGX_DIRECT_CONF | NGX_CONF_TAKE1,
ngx_conf_set_str_slot,
0,
offsetof(ngx_core_conf_t, pid),
NULL },
{ ngx_string("lock_file"),
NGX_MAIN_CONF | NGX_DIRECT_CONF | NGX_CONF_TAKE1,
ngx_conf_set_str_slot,
0,
offsetof(ngx_core_conf_t, lock_file),
NULL },
{ ngx_string("worker_processes"),
NGX_MAIN_CONF | NGX_DIRECT_CONF | NGX_CONF_TAKE1,
ngx_set_worker_processes,
0,
0,
NULL },
{ ngx_string("debug_points"),
NGX_MAIN_CONF | NGX_DIRECT_CONF | NGX_CONF_TAKE1,
ngx_conf_set_enum_slot,
0,
offsetof(ngx_core_conf_t, debug_points),
&ngx_debug_points },
{ ngx_string("user"),
NGX_MAIN_CONF | NGX_DIRECT_CONF | NGX_CONF_TAKE12,
ngx_set_user,
0,
0,
NULL },
{ ngx_string("worker_priority"),
NGX_MAIN_CONF | NGX_DIRECT_CONF | NGX_CONF_TAKE1,
ngx_set_priority,
0,
0,
NULL },
{ ngx_string("worker_cpu_affinity"),
NGX_MAIN_CONF | NGX_DIRECT_CONF | NGX_CONF_1MORE,
ngx_set_cpu_affinity,
0,
0,
NULL },
{ ngx_string("worker_rlimit_nofile"),
NGX_MAIN_CONF | NGX_DIRECT_CONF | NGX_CONF_TAKE1,
ngx_conf_set_num_slot,
0,
offsetof(ngx_core_conf_t, rlimit_nofile),
NULL },
{ ngx_string("worker_rlimit_core"),
NGX_MAIN_CONF | NGX_DIRECT_CONF | NGX_CONF_TAKE1,
ngx_conf_set_off_slot,

```



```
    0,
    offsetof(ngx_core_conf_t, rlimit_core),
    NULL },
    { ngx_string("worker_shutdown_timeout"),
    NGX_MAIN_CONF | NGX_DIRECT_CONF | NGX_CONF_TAKE1,
    ngx_conf_set_msec_slot,
    0,
    offsetof(ngx_core_conf_t, shutdown_timeout),
    NULL },
    { ngx_string("working_directory"),
    NGX_MAIN_CONF | NGX_DIRECT_CONF | NGX_CONF_TAKE1,
    ngx_conf_set_str_slot,
    0,
    offsetof(ngx_core_conf_t, working_directory),
    NULL },
    { ngx_string("env"),
    NGX_MAIN_CONF | NGX_DIRECT_CONF | NGX_CONF_TAKE1,
    ngx_set_env,
    0,
    0,
    NULL },
    { ngx_string("load_module"),
    NGX_MAIN_CONF | NGX_DIRECT_CONF | NGX_CONF_TAKE1,
    ngx_load_module,
    0,
    0,
    NULL },
    ngx_null_command
};
```

create_conf

```
static void *
ngx_event_core_create_conf(ngx_cycle_t *cycle)
{
    ngx_event_conf_t *ecf;

    ecf = ngx_palloc(cycle->pool, sizeof(ngx_event_conf_t));
    if (ecf == NULL) {
        return NULL;
    }

    ecf->connections = NGX_CONF_UNSET_UINT;
    ecf->use = NGX_CONF_UNSET_UINT;
    ecf->multi_accept = NGX_CONF_UNSET;
    ecf->accept_mutex = NGX_CONF_UNSET;
    ecf->accept_mutex_delay = NGX_CONF_UNSET_MSEC;
    ecf->name = (void *) NGX_CONF_UNSET;

#ifdef NGX_DEBUG

    if (ngx_array_init(&ecf->debug_connection, cycle->pool, 4,
        sizeof(ngx_cidr_t)) == NGX_ERROR)
    {
        return NULL;
    }

#endif

    return ecf;
}
```

```
events {
    #master调度用户请求至个worker进程时使用的负载均衡锁: on表示能让多个worker轮流地、
    序列化的响应新请求
    accept_mutex {off|on}

    #延迟等待时间, 默认为500ms
    accept_mutex_delay time;

    #accept_mutex用到的锁文件路径
    lock_file file;

    #指明使用的时间模型: 建议让Nginx自行选择
    use [epoll|rtsig|select|poll];

    #单个worker进程打开的最大并发连接数, worker_processes*worker_connections
    worker_connections 2048;
```

```
#告诉nginx收到一个新链接通知后接受尽可能多的链接
multi_accept on;
}
```

事件驱动模块在create_conf阶段,只是初始化了事件对应的连接个数等参数。

init_conf

```
static char * ngx_event_core_init_conf(ngx_cycle_t *cycle, void *conf){
    ngx_event_conf_t *ecf = conf;
    if (module == NULL) {
        for (i = 0; cycle->modules[i]; i++) {

            if (cycle->modules[i]->type != NGX_EVENT_MODULE) {
                continue;
            }

            event_module = cycle->modules[i]->ctx;

            if (ngx_strcmp(event_module->name->data, event_core_name.data)
== 0)
            {
                continue;
            }

            module = cycle->modules[i];
            break;
        }
    }

    if (module == NULL) {
        ngx_log_error(NGX_LOG_EMERG, cycle->log, 0, "no events module
found");
        return NGX_CONF_ERROR;
    }

    ngx_conf_init_uint_value(ecf->connections, DEFAULT_CONNECTIONS);
    cycle->connection_n = ecf->connections;

    ngx_conf_init_uint_value(ecf->use, module->ctx_index);

    event_module = module->ctx;
    ngx_conf_init_ptr_value(ecf->name, event_module->name->data);

    ngx_conf_init_value(ecf->multi_accept, 0);
    ngx_conf_init_value(ecf->accept_mutex, 0);
    ngx_conf_init_msec_value(ecf->accept_mutex_delay, 500);
}
```

```

    return NGX_CONF_OK;
}

```

init_module

ngx_event_module_init 在 ngx_init_cycle 里调用，fork 子进程之前，创建共享内存，存放负载均衡锁和统计用的原子变量。

```

static ngx_int_t
ngx_event_module_init(ngx_cycle_t *cycle){
    void            ***cf;
    u_char          *shared;
    size_t          size, cl;
    ngx_shm_t       shm;
    ngx_time_t      *tp;
    ngx_core_conf_t *ccf;
    ngx_event_conf_t *ecf;

    // events模块的配置结构体
    cf = ngx_get_conf(cycle->conf_ctx, ngx_events_module);

    // event_core模块的配置结构体
    ecf = (*cf)[ngx_event_core_module.ctx_index];

    if (!ngx_test_config && ngx_process <= NGX_PROCESS_MASTER) {
        ngx_log_error(NGX_LOG_NOTICE, cycle->log, 0,
            "using the \"%s\" event method", ecf->name);
    }

    // core模块的配置结构体
    ccf = (ngx_core_conf_t *) ngx_get_conf(cycle->conf_ctx,
    ngx_core_module);

    // 获取核心配置的时间精度，用在epoll里更新缓存时间
    ngx_timer_resolution = ccf->timer_resolution;

    // unix专用代码，可打开的最多文件描述符
#ifdef !NGX_WIN32
    {
        ngx_int_t      limit;
        struct rlimit   rlmt;

        // 系统调用getrlimit, Linux内核对进程的限制
        // RLIMIT_NOFILE, 进程可打开的最大文件描述符数量，超出将产生EMFILE错误
        if (getrlimit(RLIMIT_NOFILE, &rlmt) == -1) {

            // 系统调用失败则记录alert级别日志
            ngx_log_error(NGX_LOG_ALERT, cycle->log, ngx_errno,
                "getrlimit(RLIMIT_NOFILE) failed, ignored");
        }
    }
#endif
}

```

```

    } else {
        // 成功获取内核参数
        //
        // rlimt.rlim_cur是系统的软限制
        // event里配置的连接数不能超过系统内核限制
        // 或者是配置的rlimit_nofile限制
        if (ecf->connections > (ngx_uint_t) rlimt.rlim_cur
            && (ccf->rlimit_nofile == NGX_CONF_UNSET
                || ecf->connections > (ngx_uint_t) ccf->rlimit_nofile))
        {
            // 如果超过了报警告级别日志
            // limit就是上限
            limit = (ccf->rlimit_nofile == NGX_CONF_UNSET) ?
                    (ngx_int_t) rlimt.rlim_cur : ccf->rlimit_nofile;

            ngx_log_error(NGX_LOG_WARN, cycle->log, 0,
                          "%ui worker_connections exceed "
                          "open file resource limit: %i",
                          ecf->connections, limit);
        }
    }
}
}

#endif /* !(NGX_WIN32) */

// 如果非master/worker进程，即只启动一个进程，那么就没必要使用负载均衡锁
if (ccf->master == 0) {
    return NGX_OK;
}

// 已经有了负载均衡锁，已经初始化过了，就没必要再做操作
if (ngx_accept_mutex_ptr) {
    return NGX_OK;
}

// cl是一个基本长度，可以容纳原子变量
// 对齐到cache line，操作更快
cl = 128;

// 最基本的三个：负载均衡锁，连接计数器，
size = cl          /* ngx_accept_mutex */
    + cl          /* ngx_connection_counter */
    + cl;         /* ngx_temp_number */

// 创建共享内存，存放负载均衡锁和统计用的原子变量
shm.size = size;
ngx_str_set(&shm.name, "nginx_shared_zone");
shm.log = cycle->log;

// 利用 mmap 分配一块共享内存

```

```
    if (ngx_shm_alloc(&shm) != NGX_OK) {
        return NGX_ERROR;
    }

    // shared是共享内存的地址指针
    shared = shm.addr;

    // 第一个就是负载均衡锁
    ngx_accept_mutex_ptr = (ngx_atomic_t *) shared;

    // spin是-1则不使用信号量
    // 只会自旋，不会导致进程睡眠等待
    // 这样避免抢accept锁时的性能降低
    ngx_accept_mutex.spin = (ngx_uint_t) -1;

    // 初始化互斥锁
    // spin是-1则不使用信号量
    // 只会自旋，不会导致进程睡眠等待
    if (ngx_shmtx_create(&ngx_accept_mutex, (ngx_shmtx_sh_t *) shared,
                        cycle->lock_file.data) != NGX_OK){
        return NGX_ERROR;
    }

    // 连接计数器
    ngx_connection_counter = (ngx_atomic_t *) (shared + 1 * cl);

    //原子操作，把 ngx_connection_counter 从0设置为1
    (void) ngx_atomic_cmp_set(ngx_connection_counter, 0, 1);

    // 临时文件用
    ngx_temp_number = (ngx_atomic_t *) (shared + 2 * cl);

    tp = ngx_timeofday();

    // 随机数
    // 每个进程不同
    ngx_random_number = (tp->msec << 16) + ngx_pid;

    return NGX_OK;
}
```

ngx_event_process_init

ngx_event_process_init 是在 fork() 之后，worker进程初始化时调用，即每个worker里都会执行。初始化两个延后处理的事件队列、初始化定时器红黑树、发送定时信号、更新时间用、初始化cycle里的连接和事件数组、设置接受连接的回调函数为 ngx_event_accept，可以接受连接。

初始化后的ngx_connection_t连接池如下：

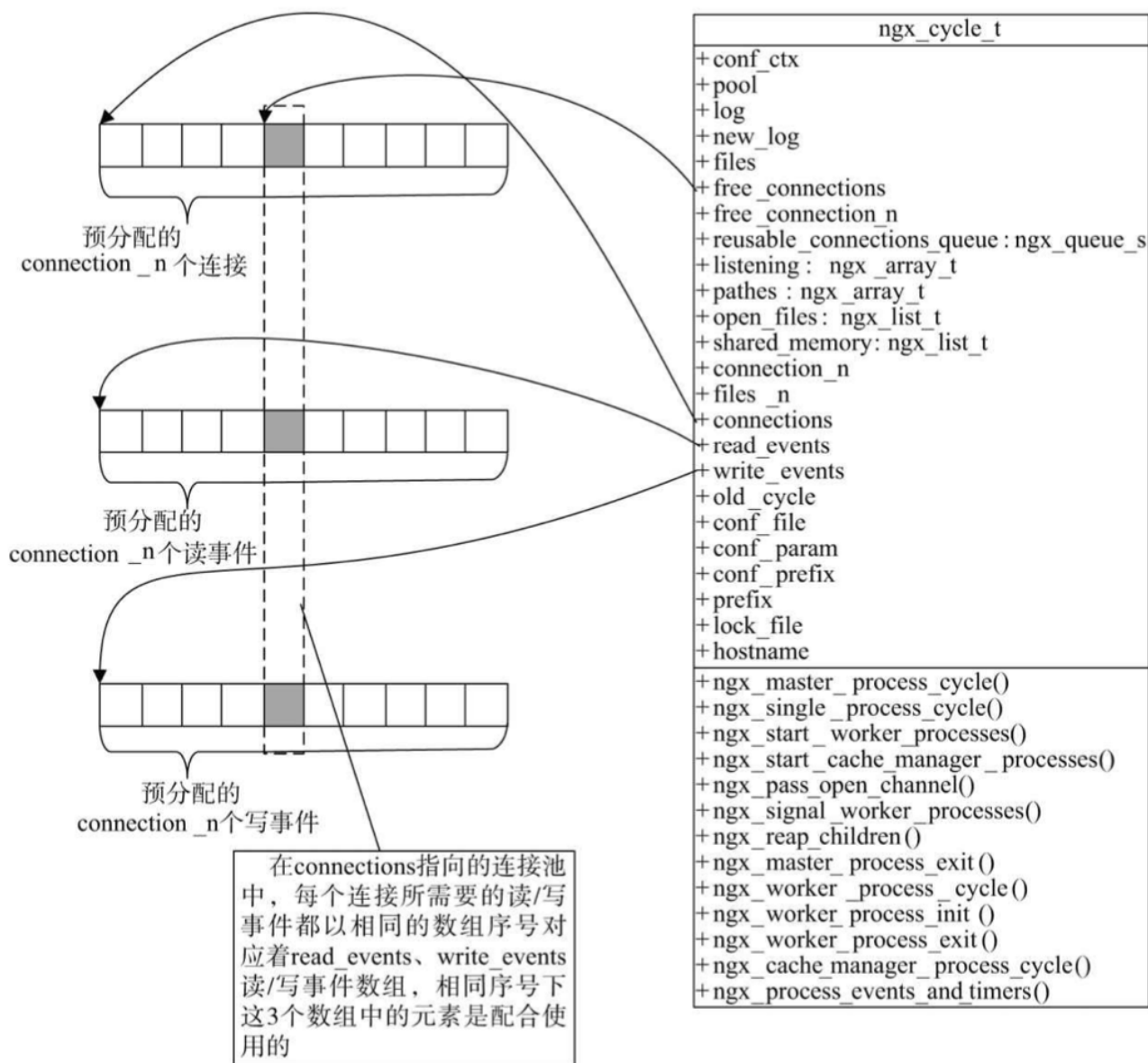


图9-1 ngx_connection_t连接池示意图

```

static ngx_int_t
ngx_event_process_init(ngx_cycle_t *cycle)
{
    ngx_uint_t          m, i;
    ngx_event_t         *rev, *wev;
    ngx_listening_t     *ls;
    ngx_connection_t     *c, *next, *old;
    ngx_core_conf_t      *ccf;
    ngx_event_conf_t     *ecf;
    ngx_event_module_t   *module;

    // core模块的配置结构体
    ccf = (ngx_core_conf_t *) ngx_get_conf(cycle->conf_ctx,
    ngx_core_module);

    // event_core模块的配置结构体

```

```
ecf = ngx_event_get_conf(cycle->conf_ctx, ngx_event_core_module);

// 使用master/worker多进程, 使用负载均衡
if (ccf->master && ccf->worker_processes > 1 && ecf->accept_mutex) {

    // 设置全局变量
    // 使用负载均衡, 刚开始未持有锁, 设置抢锁的间隔等待时间
    ngx_use_accept_mutex = 1;
    ngx_accept_mutex_held = 0;
    ngx_accept_mutex_delay = ecf->accept_mutex_delay;

} else {
    // 单进程、未明确指定负载均衡, 就不使用负载均衡
    ngx_use_accept_mutex = 0;
}

// 初始化两个延后处理的事件队列
// 如果使用了reuseport, 或者不使用负载均衡
// 那么这两个队列就完全不会用到
ngx_queue_init(&ngx_posted_accept_events);
ngx_queue_init(&ngx_posted_events);

// 初始化定时器红黑树
if (ngx_event_timer_init(cycle->log) == NGX_ERROR) {
    return NGX_ERROR;
}

// 遍历事件模块, 但只执行实际使用的事件模块对应初始化函数比如 ngx_epoll_module
for (m = 0; cycle->modules[m]; m++) {
    if (cycle->modules[m]->type != NGX_EVENT_MODULE) {
        continue;
    }

    // 找到use指令使用的事件模型, 或者是默认事件模型
    if (cycle->modules[m]->ctx_index != ecf->use) {
        continue;
    }

    module = cycle->modules[m]->ctx;

    // 调用事件模块的事件初始化函数比如 ngx_epoll_init
    // 调用epoll_create初始化epoll机制
    // 设置全局变量, 操作系统提供的底层数据收发接口
    // 初始化全局的事件模块访问接口, 指向epoll的函数
    // 默认使用et模式, 边缘触发, 高速
    if (module->actions.init(cycle, ngx_timer_resolution) != NGX_OK) {
        exit(2);
    }

    break;
}
```



```
// NGX_USE_TIMER_EVENT标志量 epoll 无此标志位
// ngx_timer_resolution = ccf->timer_resolution;默认值是0
// 所以只有使用了timer_resolution指令才会发信号
if (ngx_timer_resolution && !(ngx_event_flags & NGX_USE_TIMER_EVENT)) {
    struct sigaction sa;
    struct itimerval itv;

    // 设置信号掩码, sigalarm
    ngx_memzero(&sa, sizeof(struct sigaction));
    sa.sa_handler = ngx_timer_signal_handler;
    sigemptyset(&sa.sa_mask);

    if (sigaction(SIGALRM, &sa, NULL) == -1) {
        ngx_log_error(NGX_LOG_ALERT, cycle->log, ngx_errno,
            "sigaction(SIGALRM) failed");
        return NGX_ERROR;
    }

    // 设置信号发送的时间间隔, 也就是nginx的时间精度
    // 收到信号会设置设置ngx_event_timer_alarm变量
    // 在epoll的ngx_epoll_process_events里检查, 更新时间的标志
    itv.it_interval.tv_sec = ngx_timer_resolution / 1000;
    itv.it_interval.tv_usec = (ngx_timer_resolution % 1000) * 1000;
    itv.it_value.tv_sec = ngx_timer_resolution / 1000;
    itv.it_value.tv_usec = (ngx_timer_resolution % 1000) * 1000;

    if (setitimer(ITIMER_REAL, &itv, NULL) == -1) {
        ngx_log_error(NGX_LOG_ALERT, cycle->log, ngx_errno,
            "setitimer() failed");
    }
}

cycle->connections =
    ngx_alloc(sizeof(ngx_connection_t) * cycle->connection_n, cycle->log);
if (cycle->connections == NULL) {
    return NGX_ERROR;
}

c = cycle->connections;

cycle->read_events = ngx_alloc(sizeof(ngx_event_t) * cycle->connection_n,
    cycle->log);
if (cycle->read_events == NULL) {
    return NGX_ERROR;
}

rev = cycle->read_events;
for (i = 0; i < cycle->connection_n; i++) {
    rev[i].closed = 1;
    rev[i].instance = 1;
}
```

```
cycle->write_events = ngx_alloc(sizeof(ngx_event_t) * cycle->connection_n,
                                cycle->log);
if (cycle->write_events == NULL) {
    return NGX_ERROR;
}

wev = cycle->write_events;
for (i = 0; i < cycle->connection_n; i++) {
    wev[i].closed = 1;
}

i = cycle->connection_n;
next = NULL;

do {
    i--;

    c[i].data = next;
    c[i].read = &cycle->read_events[i];
    c[i].write = &cycle->write_events[i];
    c[i].fd = (ngx_socket_t) -1;

    next = &c[i];
} while (i);

cycle->free_connections = next;
cycle->free_connection_n = cycle->connection_n;

/* for each listening socket */

ls = cycle->listening.elts;
for (i = 0; i < cycle->listening.nelts; i++) {
    c = ngx_get_connection(ls[i].fd, cycle->log);

    if (c == NULL) {
        return NGX_ERROR;
    }

    c->type = ls[i].type;
    c->log = &ls[i].log;

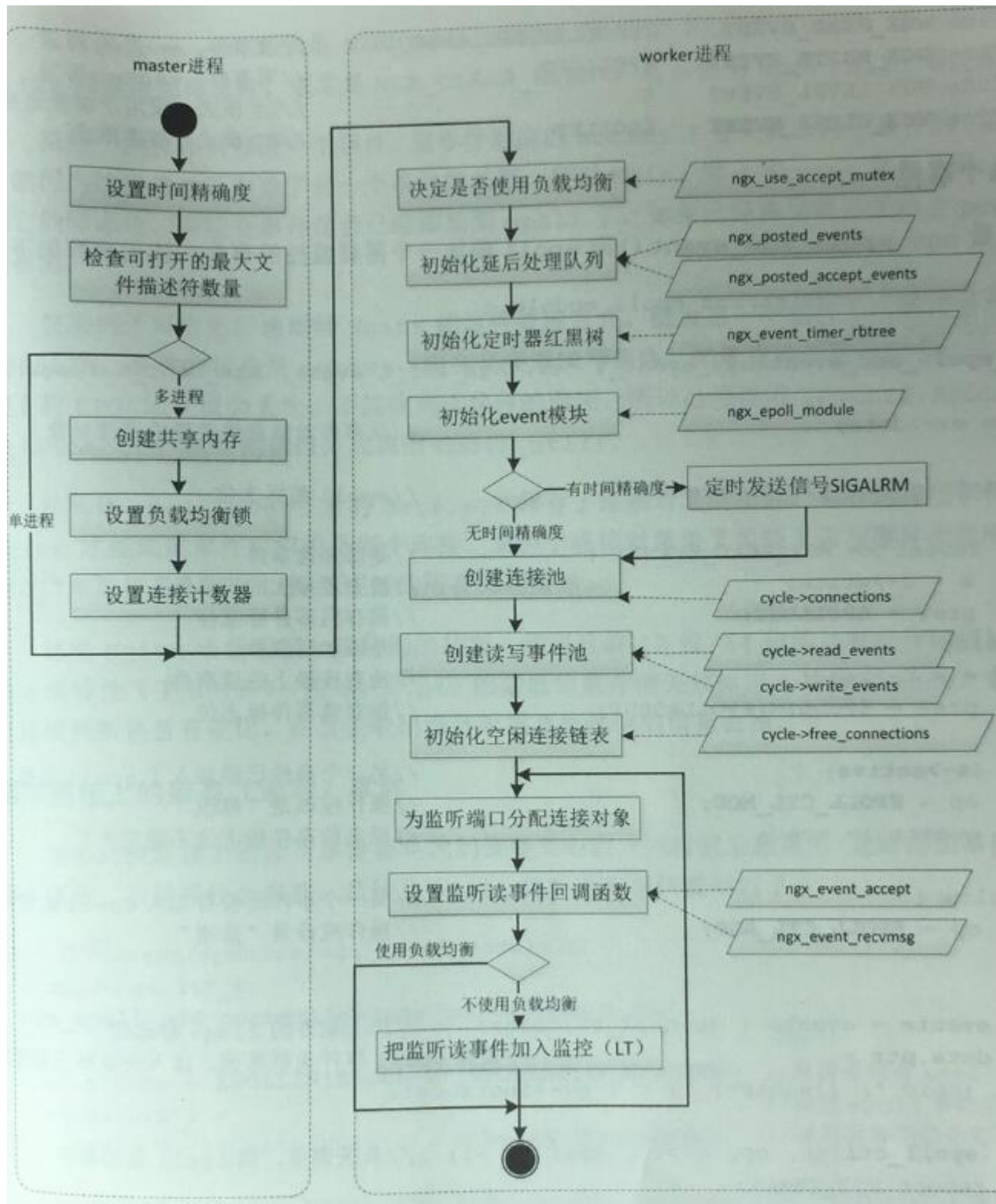
    c->listening = &ls[i];
    ls[i].connection = c;

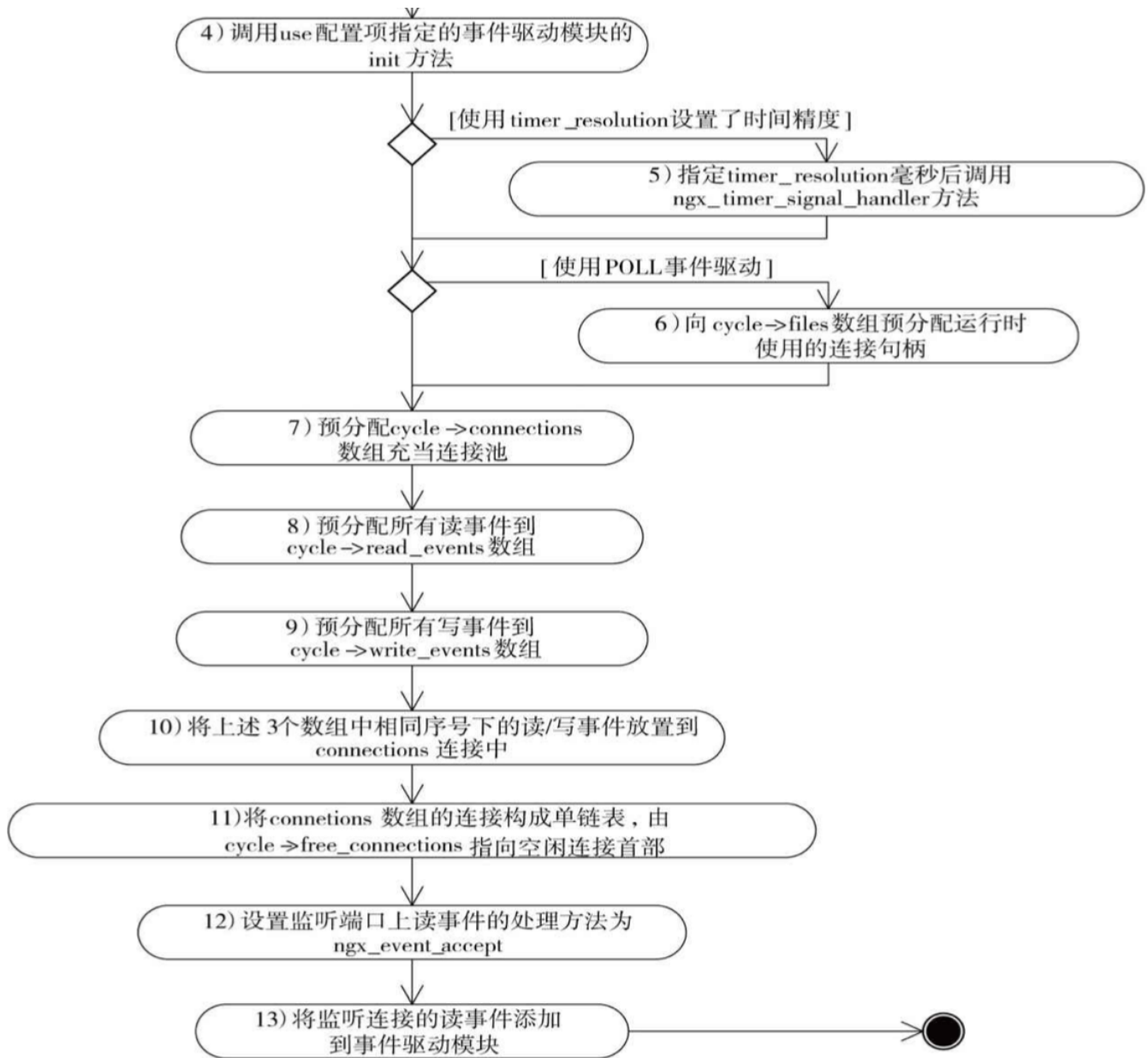
    rev = c->read;

    rev->log = c->log;
    rev->accept = 1;

    if (!(ngx_event_flags & NGX_USE_IOCP_EVENT)) {
```

```
    if (ls[i].previous) {  
  
        /*  
         * delete the old accept events that were bound to  
         * the old cycle read events array  
         */  
  
        old = ls[i].previous->connection;  
  
        if (ngx_del_event(old->read, NGX_READ_EVENT, NGX_CLOSE_EVENT)  
            == NGX_ERROR)  
        {  
            return NGX_ERROR;  
        }  
  
        old->fd = (ngx_socket_t) -1;  
    }  
}
```





epoll事件驱动模块(ngx_epoll_module)

epoll介绍

<https://zhuanlan.zhihu.com/p/63179839>

ngx_epoll_module模块

```

static ngx_event_module_t  ngx_epoll_module_ctx = {
    &epoll_name,
    ngx_epoll_create_conf,          /* create configuration */
    ngx_epoll_init_conf,           /* init configuration */
    {
        ngx_epoll_add_event,        /* add an event */
        ngx_epoll_del_event,        /* delete an event */
        ngx_epoll_add_event,        /* enable an event */
        ngx_epoll_del_event,        /* disable an event */
        ngx_epoll_add_connection,   /* add an connection */
        ngx_epoll_del_connection,   /* delete an connection */
#if (NGX_HAVE_EVENTFD)
        ngx_epoll_notify,           /* trigger a notify */
#else
        NULL,                       /* trigger a notify */
#endif
        ngx_epoll_process_events,    /* process the events */
        ngx_epoll_init,              /* init the events */
        ngx_epoll_done,              /* done the events */
    }
};

ngx_module_t  ngx_epoll_module = {
    NGX_MODULE_V1,
    &ngx_epoll_module_ctx,          /* module context */
    ngx_epoll_commands,             /* module directives */
    NGX_EVENT_MODULE,               /* module type */
    NULL,                           /* init master */
    NULL,                           /* init module */
    NULL,                           /* init process */
    NULL,                           /* init thread */
    NULL,                           /* exit thread */
    NULL,                           /* exit process */
    NULL,                           /* exit master */
    NGX_MODULE_V1_PADDING
};

```

```

static ngx_command_t  ngx_epoll_commands[] = {
    //这个配置项表示调用一次epoll_wait最多可以返回的事件数
    { ngx_string("epoll_events"),
      NGX_EVENT_CONF|NGX_CONF_TAKE1,

```

```

    ngx_conf_set_num_slot,
    0,
    offsetof(ngx_epoll_conf_t, events),
    NULL },
    //指明在开启异步I/O且使用io_setup系统调用初始化异步I/O上下文环境时, 初始分配的异步
    I/O事件个数
    { ngx_string("worker_aio_requests"),
    NGX_EVENT_CONF|NGX_CONF_TAKE1,
    ngx_conf_set_num_slot,
    0,
    offsetof(ngx_epoll_conf_t, aio_requests),
    NULL },

    ngx_null_command
};

```

- ngx_epoll_init

```

/*初始化放在: ngx_event_process_init中进行*/

static int                ep = -1; //epoll对象描述符
static struct epoll_event *event_list; //链表
static ngx_uint_t         nevents; //链表应该分配的大小

static ngx_int_t
ngx_epoll_init(ngx_cycle_t *cycle, ngx_msec_t timer)
{
    ngx_epoll_conf_t *epcf; //存储配置项的结构体

    epcf = ngx_event_get_conf(cycle->conf_ctx, ngx_epoll_module); //获取解析所得的配置项的值

    if (ep == -1) {
        //创建epoll对象, 在很多Linux内核版本中, 并不处理这个函数的参数
        ep = epoll_create(cycle->connection_n / 2);

        //创建失败
        if (ep == -1) {
            ngx_log_error(NGX_LOG_EMERG, cycle->log, ngx_errno,
                          "epoll_create() failed");
            return NGX_ERROR;
        }
    }

    #if (NGX_HAVE_FILE_AIO)
        //异步I/O
        ngx_epoll_aio_init(cycle, epcf);
    #endif
}

```

```

//解析配置项所得的epoll_wait一次可最多返回的时间个数较大
if (nevents < epcf->events) {
    if (event_list) {
        //销毁原来的存储返回事件的链表
        ngx_free(event_list);
    }

    //重新分配链表空间
    event_list = ngx_alloc(sizeof(struct epoll_event) * epcf-
>events,
                                cycle->log);
    if (event_list == NULL) {
        return NGX_ERROR;
    }
}

//更新参数
nevents = epcf->events;

ngx_io = ngx_os_io;

//Nginx时间框架处理事件时封装的接口
ngx_event_actions = ngx_epoll_module_ctx.actions;

#ifdef NGX_HAVE_CLEAR_EVENT
    //默认采用E模式使用epoll, NGX_HAVE_CLEAR_EVENT宏实际上就是在告诉nginx使用
    ET模式
    ngx_event_flags = NGX_USE_CLEAR_EVENT
#else
    ngx_event_flags = NGX_USE_LEVEL_EVENT
#endif
                    |NGX_USE_GREEDY_EVENT
                    |NGX_USE_EPOLL_EVENT;

    return NGX_OK;
}

```

- ngx_epoll_add_event

```

static ngx_int_t
ngx_epoll_add_event(ngx_event_t *ev, ngx_int_t event, ngx_uint_t
flags)
{
    int                op;
    uint32_t           events, prev;
    ngx_event_t        *e;
    ngx_connection_t    *c;
    struct epoll_event  ee;

```



```
//每个事件的数据成员存放着其对应的ngx_connection_t连接
c = ev->data;

//下面会根据event参数确定当前事件是读事件还是写事件，这会决定events是加上
EPOLLIN标志还是EPOLLOUT标志位
events = (uint32_t) event;

//关于下面这段代码的解释：需要结合后面的依据active标志位确定是否为活跃事件的代码来看
/*首先，
man epoll:
Q: What happens if you register the same file descriptor on an
epoll instance twice?

A: You will probably get EEXIST. However, it is possible to
add a duplicate (dup(2), dup2(2), fcntl(2) F_DUPFD) descriptor
to the same epoll instance. This can be a useful technique for
filtering events, if the duplicate file descriptors are registered
with different events masks.*/

/*所以nginx这里就是为了避免这种情况，当要在epoll中加入对一个fd读事件(即
NGX_READ_EVENT)的监听时，nginx先看一下与这个fd相关的写事件的状态，即e=c-
>write，如果此时e->active为1，说明该fd之前已经以NGX_WRITE_EVENT方式被加到
epoll中了，此时只需要使用mod方式，将我们的需求加进去，否则才使用add方式，将该fd注
册到epoll中。反之处理NGX_WRITE_EVENT时道理是一样的。*/

    if (event == NGX_READ_EVENT) {
        e = c->write;
        prev = EPOLLOUT;
    #if (NGX_READ_EVENT != EPOLLIN)
        events = EPOLLIN;
    #endif

    } else {
        e = c->read;
        prev = EPOLLIN;
    #if (NGX_WRITE_EVENT != EPOLLOUT)
        events = EPOLLOUT;
    #endif
    }

//依据active标志位确定是否为活跃事件
if (e->active) {
    op = EPOLL_CTL_MOD; //是活跃事件，修改事件
    events |= prev;

} else {
    op = EPOLL_CTL_ADD; //不是活跃事件，添加事件
}
```

```

ee.events = events | (uint32_t) flags; // 设定events标志位
// ptr存储的是ngx_connection_t连接, instance是过期事件标志位
ee.data.ptr = (void *) ((uintptr_t) c | ev->instance);

ngx_log_debug3(NGX_LOG_DEBUG_EVENT, ev->log, 0,
               "epoll add event: fd:%d op:%d ev:%08XD",
               c->fd, op, ee.events);

if (epoll_ctl(ep, op, c->fd, &ee) == -1) {
    ngx_log_error(NGX_LOG_ALERT, ev->log, ngx_errno,
                  "epoll_ctl(%d, %d) failed", op, c->fd);
    return NGX_ERROR;
}

ev->active = 1;
#if 0
    ev->oneshot = (flags & NGX_ONESHOT_EVENT) ? 1 : 0;
#endif

    return NGX_OK;
}

```

- ngx_epoll_process_events

```

static ngx_int_t
ngx_epoll_process_events(ngx_cycle_t *cycle, ngx_msec_t timer,
ngx_uint_t flags)
{
    int                events;
    uint32_t           revents;
    ngx_int_t          instance, i;
    ngx_uint_t         level;
    ngx_err_t          err;
    ngx_event_t        *rev, *wev, **queue;
    ngx_connection_t   *c;

    /* NGX_TIMER_INFINITE == INFTIM */

    ngx_log_debug1(NGX_LOG_DEBUG_EVENT, cycle->log, 0,
                   "epoll timer: %M", timer);

    // 调用epoll_wait获取事件
    events = epoll_wait(ep, event_list, (int) nevents, timer);

    err = (events == -1) ? ngx_errno : 0;

    // 更新时间
    if (flags & NGX_UPDATE_TIME || ngx_event_timer_alarm) {
        ngx_time_update();
    }
}

```

```

}

//epoll_wait出错处理
if (err) {
    if (err == NGX_EINTR) {

        if (ngx_event_timer_alarm) {
            ngx_event_timer_alarm = 0;
            return NGX_OK;
        }

        level = NGX_LOG_INFO;

    } else {
        level = NGX_LOG_ALERT;
    }

    ngx_log_error(level, cycle->log, err, "epoll_wait() failed");
    return NGX_ERROR;
}

//本次调用没有事件发生
if (events == 0) {
    if (timer != NGX_TIMER_INFINITE) {
        return NGX_OK;
    }

    ngx_log_error(NGX_LOG_ALERT, cycle->log, 0,
        "epoll_wait() returned no events without
timeout");
    return NGX_ERROR;
}

ngx_mutex_lock(ngx_posted_events_mutex);

//遍历本次epoll_wait返回的所有事件
for (i = 0; i < events; i++) {
    //获取连接ngx_connection_t的地址
    c = event_list[i].data.ptr;

    //连接的地址最后一位具有特殊意义：用于存储instance变量，将其取出来
    instance = (uintptr_t) c & 1;
    //无论是32位还是64位机器，其地址最后一位一定是0，获取真正地址
    c = (ngx_connection_t *) ((uintptr_t) c & (uintptr_t) ~1);
    //取出读事件
    rev = c->read;

    //判断读事件是否为过期事件
    if (c->fd == -1 || rev->instance != instance) {

        /*

```

```

        * the stale event from a file descriptor
        * that was just closed in this iteration
        */

        ngx_log_debug1(NGX_LOG_DEBUG_EVENT, cycle->log, 0,
                        "epoll: stale event %p", c);
        continue;
    }

    //取出事件类型
    revents = event_list[i].events;

    ngx_log_debug3(NGX_LOG_DEBUG_EVENT, cycle->log, 0,
                    "epoll: fd:%d ev:%04XD d:%p",
                    c->fd, revents, event_list[i].data.ptr);

    if (revents & (EPOLLERR|EPOLLHUP)) {
        ngx_log_debug2(NGX_LOG_DEBUG_EVENT, cycle->log, 0,
                        "epoll_wait() error on fd:%d ev:%04XD",
                        c->fd, revents);
    }

    if ((revents & (EPOLLERR|EPOLLHUP))
        && (revents & (EPOLLIN|EPOLLOUT)) == 0)
    {
        /*
         * if the error events were returned without EPOLLIN or
EPOLLOUT,
         * then add these flags to handle the events at least in
one
         * active handler
         */

        revents |= EPOLLIN|EPOLLOUT;
    }

    //是读事件且该事件是活跃的
    if ((revents & EPOLLIN) && rev->active) {

        if ((flags & NGX_POST_THREAD_EVENTS) && !rev->accept) {
            rev->posted_ready = 1;

        } else {
            rev->ready = 1;
        }

        //事件需要延后处理
        if (flags & NGX_POST_EVENTS) {
            /*如果要在post队列中延后处理该事件，首先要判断它是新连接时间还是
普通事件
            以确定是把它加入到ngx_posted_accept_events队列或者

```

```
ngx_posted_events队列中。*/
    queue = (ngx_event_t **) (rev->accept ?
        &ngx_posted_accept_events :
&ngx_posted_events);
    //将该事件添加到相应的延后队列中
    ngx_locked_post_event(rev, queue);尽可能accept请求

    } else {
        //立即调用事件回调方法来处理这个事件
        rev->handler(rev);
    }
}

wev = c->write;

if ((revents & EPOLLOUT) && wev->active) {

    if (c->fd == -1 || wev->instance != instance) {

        /*
        * the stale event from a file descriptor
        * that was just closed in this iteration
        */

        ngx_log_debug1(NGX_LOG_DEBUG_EVENT, cycle->log, 0,
            "epoll: stale event %p", c);
        continue;
    }

    if (flags & NGX_POST_THREAD_EVENTS) {
        wev->posted_ready = 1;
    } else {
        wev->ready = 1;
    }

    if (flags & NGX_POST_EVENTS) {
        ngx_locked_post_event(wev, &ngx_posted_events);
    } else {
        wev->handler(wev);
    }
}

}

ngx_mutex_unlock(ngx_posted_events_mutex);

return NGX_OK;
}
```

过期事件的产生：

`instance`标志位为什么可以判断事件是否过期？从上面的代码可以看出，`instance`标志位的使用其实很简单，它利用了指针的最后一位一定是0这一特性。既然最后一位始终都是0，那么不如用来表示`instance`。这样，在使用`ngx_epoll_add_event`方法向`epoll`中添加事件时，就把`epoll_event`中联合成员`data`的`ptr`成员指向`ngx_connection_t`连接的地址，同时把最后一位置为这个事件的`instance`标志。而在`ngx_epoll_process_events`方法中取出指向连接的`ptr`地址时，先把最后一位`instance`取出来，再把`ptr`还原成正常的地址赋给`ngx_connection_t`连接。这样，`instance`究竟放在何处的的问题也就解决了。

那么，过期事件又是怎么回事呢？举个例子，假设`epoll_wait`一次返回3个事件，在第1个事件的处理过程中，由于业务的需要，所以关闭了一个连接，而这个连接恰好对应第3个事件。这样的话，在处理到第3个事件时，这个事件就已经是过期事件了，一旦处理必然出错。既然如此，把关闭的这个连接的`fd`套接字置为-1能解决问题吗？答案是不能处理所有情况。

下面先来看看这种貌似不可能发生的场景到底是怎么发生的：假设第3个事件对应的`ngx_connection_t`连接中的`fd`套接字原先是50，处理第1个事件时把这个连接的套接字关闭了，同时置为-1，并且调用`ngx_free_connection`将该连接归还给连接池。在`ngx_epoll_process_events`方法的循环中开始处理第2个事件，恰好第2个事件是建立新连接事件，调用`ngx_get_connection`从连接池中取出的连接非常可能就是刚刚释放的第3个事件对应的连接。由于套接字50刚刚被释放，Linux内核非常有可能把刚刚释放的套接字50又分配给

新建立的连接。因此，在循环中处理第3个事件时，这个事件就是过期的了！它对应的事件是关闭的连接，而不是新建立的连接。

如何解决这个问题？依靠`instance`标志位。当调用`ngx_get_connection`从连接池中获取一个新连接时，`instance`标志位就会置反，代码如下所示。

- 为了讲清楚上面为啥会有过期事件稍微扫一下`ngx_get_connection`方法，后面http模块还会看到这个方法

```
ngx_connection_t *
ngx_get_connection(ngx_socket_t s, ngx_log_t *log)
{
    ngx_uint_t      instance;
    ngx_event_t     *rev, *wev;
    ngx_connection_t *c;
```

```
/* disable warning: Win32 SOCKET is u_int while UNIX socket is int
*/

if (ngx_cycle->files && (ngx_uint_t) s >= ngx_cycle->files_n) {
    ngx_log_error(NGX_LOG_ALERT, log, 0,
        "the new socket has number %d, "
        "but only %ui files are available",
        s, ngx_cycle->files_n);
    return NULL;
}

/* ngx_mutex_lock */

//从连接池中获取一个连接
c = ngx_cycle->free_connections;

if (c == NULL) {
    ngx_drain_connections();
    c = ngx_cycle->free_connections;
}

if (c == NULL) {
    ngx_log_error(NGX_LOG_ALERT, log, 0,
        "%ui worker_connections are not enough",
        ngx_cycle->connection_n);

    /* ngx_mutex_unlock */

    return NULL;
}

ngx_cycle->free_connections = c->data;
ngx_cycle->free_connection_n--;

/* ngx_mutex_unlock */

if (ngx_cycle->files) {
    ngx_cycle->files[s] = c;
}

//获取读事件和写事件
rev = c->read;
wev = c->write;

ngx_memzero(c, sizeof(ngx_connection_t));

c->read = rev;
c->write = wev;
c->fd = s;
c->log = log;
```

```
//获取instance标志位
instance = rev->instance;

ngx_memzero(rev, sizeof(ngx_event_t));
ngx_memzero(wev, sizeof(ngx_event_t));

//instance标志位置反
rev->instance = !instance;
wev->instance = !instance;

rev->index = NGX_INVALID_INDEX;
wev->index = NGX_INVALID_INDEX;

rev->data = c;
wev->data = c;

wev->write = 1;

return c;
}
```

事件驱动中的文件异步IO事件

在 Nginx 中，文件异步 I/O 事件完成后的通知是集成到 epoll 对象中。

```
/* 初始化文件异步IO的上下文结构 */
static int
io_setup(u_int nr_reqs, aio_context_t *ctx)
{
    return syscall(SYS_io_setup, nr_reqs, ctx);
}

/* 销毁文件异步IO的上下文结构 */
static int
io_destroy(aio_context_t ctx)
{
    return syscall(SYS_io_destroy, ctx);
}

/* 从文件异步IO操作队列中读取操作 */
static int
io_getevents(aio_context_t ctx, long min_nr, long nr, struct io_event
*events,
    struct timespec *tmo)
{
    return syscall(SYS_io_getevents, ctx, min_nr, nr, events, tmo);
}
```



```
/* 异步IO的初始化 */
static void
ngx_epoll_aio_init(ngx_cycle_t *cycle, ngx_epoll_conf_t *epcf)
{
    int                n;
    struct epoll_event ee;

    /* 使用Linux系统调用获取一个描述符句柄 */
    ngx_eventfd = syscall(SYS_eventfd, 0);

    if (ngx_eventfd == -1) {
        ngx_log_error(NGX_LOG_EMERG, cycle->log, ngx_errno,
            "eventfd() failed");
        ngx_file_aio = 0;
        return;
    }

    ngx_log_debug1(NGX_LOG_DEBUG_EVENT, cycle->log, 0,
        "eventfd: %d", ngx_eventfd);

    n = 1;

    /* 设置ngx_eventfd描述符句柄为非阻塞IO模式 */
    if (ioctl(ngx_eventfd, FIONBIO, &n) == -1) {
        ngx_log_error(NGX_LOG_EMERG, cycle->log, ngx_errno,
            "ioctl(eventfd, FIONBIO) failed");
        goto failed;
    }

    /* 初始化文件异步IO的上下文结构 */
    if (io_setup(epcf->aio_requests, &ngx_aio_ctx) == -1) {
        ngx_log_error(NGX_LOG_EMERG, cycle->log, ngx_errno,
            "io_setup() failed");
        goto failed;
    }

    /* 设置异步IO事件ngx_eventfd_event, 该事件是ngx_eventfd对应的ngx_event事件 */
    /*
    /* 用于异步IO完成通知的ngx_eventfd_event事件, 它与ngx_eventfd_conn连接对应 */
    ngx_eventfd_event.data = &ngx_eventfd_conn;
    /* 在异步IO事件完成后, 调用ngx_epoll_eventfd_handler处理方法 */
    ngx_eventfd_event.handler = ngx_epoll_eventfd_handler;
    /* 设置事件相应的日志 */
    ngx_eventfd_event.log = cycle->log;
    /* 设置active标志位 */
    ngx_eventfd_event.active = 1;
    /* 初始化ngx_eventfd_conn 连接 */
    ngx_eventfd_conn.fd = ngx_eventfd;
    /* ngx_eventfd_conn连接的读事件就是ngx_eventfd_event事件 */
    ngx_eventfd_conn.read = &ngx_eventfd_event;
*/
}
```

```

    /* 设置连接的相应日志 */
    ngx_eventfd_conn.log = cycle->log;

    ee.events = EPOLLIN|EPOLLET;
    ee.data.ptr = &ngx_eventfd_conn;

    /* 向epoll对象添加异步IO通知描述符ngx_eventfd */
    if (epoll_ctl(ep, EPOLL_CTL_ADD, ngx_eventfd, &ee) != -1) {
        return;
    }

    /* 若添加出错, 则销毁文件异步IO上下文结构, 并返回 */
    ngx_log_error(NGX_LOG_EMERG, cycle->log, ngx_errno,
        "epoll_ctl(EPOLL_CTL_ADD, eventfd) failed");

    if (io_destroy(ngx_aio_ctx) == -1) {
        ngx_log_error(NGX_LOG_ALERT, cycle->log, ngx_errno,
            "io_destroy() failed");
    }

failed:

    if (close(ngx_eventfd) == -1) {
        ngx_log_error(NGX_LOG_ALERT, cycle->log, ngx_errno,
            "eventfd close() failed");
    }

    ngx_eventfd = -1;
    ngx_aio_ctx = 0;
    ngx_file_aio = 0;
}

#endif

#if (NGX_HAVE_FILE_AIO)

/* 处理已完成的异步IO事件 */
static void
ngx_epoll_eventfd_handler(ngx_event_t *ev)
{
    int                n, events;
    long               i;
    uint64_t           ready;
    ngx_err_t          err;
    ngx_event_t        *e;
    ngx_event_aio_t    *aio;
    struct io_event     event[64];    /* 一次最多处理64个事件 */
    struct timespec     ts;

    ngx_log_debug0(NGX_LOG_DEBUG_EVENT, ev->log, 0, "eventfd handler");

```

```
/* 获取已完成的事件数，并将其设置到ready */
n = read(ngx_eventfd, &ready, 8);

err = ngx_errno;

ngx_log_debug1(NGX_LOG_DEBUG_EVENT, ev->log, 0, "eventfd: %d", n);

if (n != 8) {
    if (n == -1) {
        if (err == NGX_EAGAIN) {
            return;
        }

        ngx_log_error(NGX_LOG_ALERT, ev->log, err, "read(eventfd)
failed");
        return;
    }

    ngx_log_error(NGX_LOG_ALERT, ev->log, 0,
        "read(eventfd) returned only %d bytes", n);
    return;
}

ts.tv_sec = 0;
ts.tv_nsec = 0;

/* 遍历ready，处理异步IO事件 */
while (ready) {

    /* 获取已完成的异步IO事件 */
    events = io_getevents(ngx_aio_ctx, 1, 64, event, &ts);

    ngx_log_debug1(NGX_LOG_DEBUG_EVENT, ev->log, 0,
        "io_getevents: %l", events);

    if (events > 0) {
        ready -= events; /* ready减去已经取出的事件数 */

        /* 处理已被取出的事件 */
        for (i = 0; i < events; i++) {

            ngx_log_debug4(NGX_LOG_DEBUG_EVENT, ev->log, 0,
                "io_event: %uXL %uXL %L %L",
                event[i].data, event[i].obj,
                event[i].res, event[i].res2);

            /* 获取异步IO事件对应的实际事件 */
            e = (ngx_event_t *) (uintptr_t) event[i].data;

            e->complete = 1;
            e->active = 0;
        }
    }
}
```

```

        e->ready = 1;

        aio = e->data;
        aio->res = event[i].res;

        /* 将实际事件加入到ngx_posted_event队列中等待处理 */
        ngx_post_event(e, &ngx_posted_events);
    }

    continue;
}

if (events == 0) {
    return;
}

/* events == -1 */
ngx_log_error(NGX_LOG_ALERT, ev->log, ngx_errno,
              "io_getevents() failed");
return;
}
}

#endif

```

定时器事件

Nginx实现了自己的定时器触发机制，它与epoll等事件驱动模块处理的网络事件不同：在网络事件中，网络事件的触发是由内核完成的，内核如果支持epoll就可以使用 ngx_epoll_module模块驱动事件，内核如果仅支持select那就得使用ngx_select_module模块驱动事件；定时器事件则完全是由Nginx自身实现的，它与内核完全无关。

定时器数据结构

nginx的定时器逻辑上就是一个红黑树，针对于红黑树的几个方法来实现相关的超时检测。

1. ngx_event_timer_init 建立红黑树
2. ngx_event_add_timer 添加事件到红黑树中
3. ngx_event_del_timer 删除红黑树中的事件
4. ngx_event_find_timer 查找红黑树中最小的节点

定时器超时处理逻辑

```

//处理红黑树中所有超时的事件
void
ngx_event_expire_timers(void)
{
    ngx_event_t      *ev;

```

```

    ngx_rbtrees_node_t *node, *root, *sentinel;

    sentinel = ngx_event_timer_rbtrees.sentinel;

//死循环, 找到所有的超时的timer, 然后处理他们
    for ( ;; ) {

        ngx_mutex_lock(ngx_event_timer_mutex);

        root = ngx_event_timer_rbtrees.root;

        if (root == sentinel) {
            return;
        }

        node = ngx_rbtrees_min(root, sentinel);    //获取key最小的节点

        /* node->key <= ngx_current_time */

        if ((ngx_msec_int_t) (node->key - ngx_current_msec) <= 0) {    //判断该节点是否超时, 如果超时的话, 就执行处理函数, 否则就可以跳出循环了
            //通过偏移来获取当前timer所在的event
            ev = (ngx_event_t *) ((char *) node - offsetof(ngx_event_t, timer));

            ngx_log_debug2(NGX_LOG_DEBUG_EVENT, ev->log, 0,
                "event timer del: %d: %M",
                ngx_event_ident(ev->data), ev->timer.key);
//将当前timer移除
            ngx_rbtrees_delete(&ngx_event_timer_rbtrees, &ev->timer);

            ngx_mutex_unlock(ngx_event_timer_mutex);

            ev->timer_set = 0;

            ev->timedout = 1;

            ev->handler(ev);    //调用event的handler来处理这个事件

            continue;
        }

        break;
    }

    ngx_mutex_unlock(ngx_event_timer_mutex);
}

```

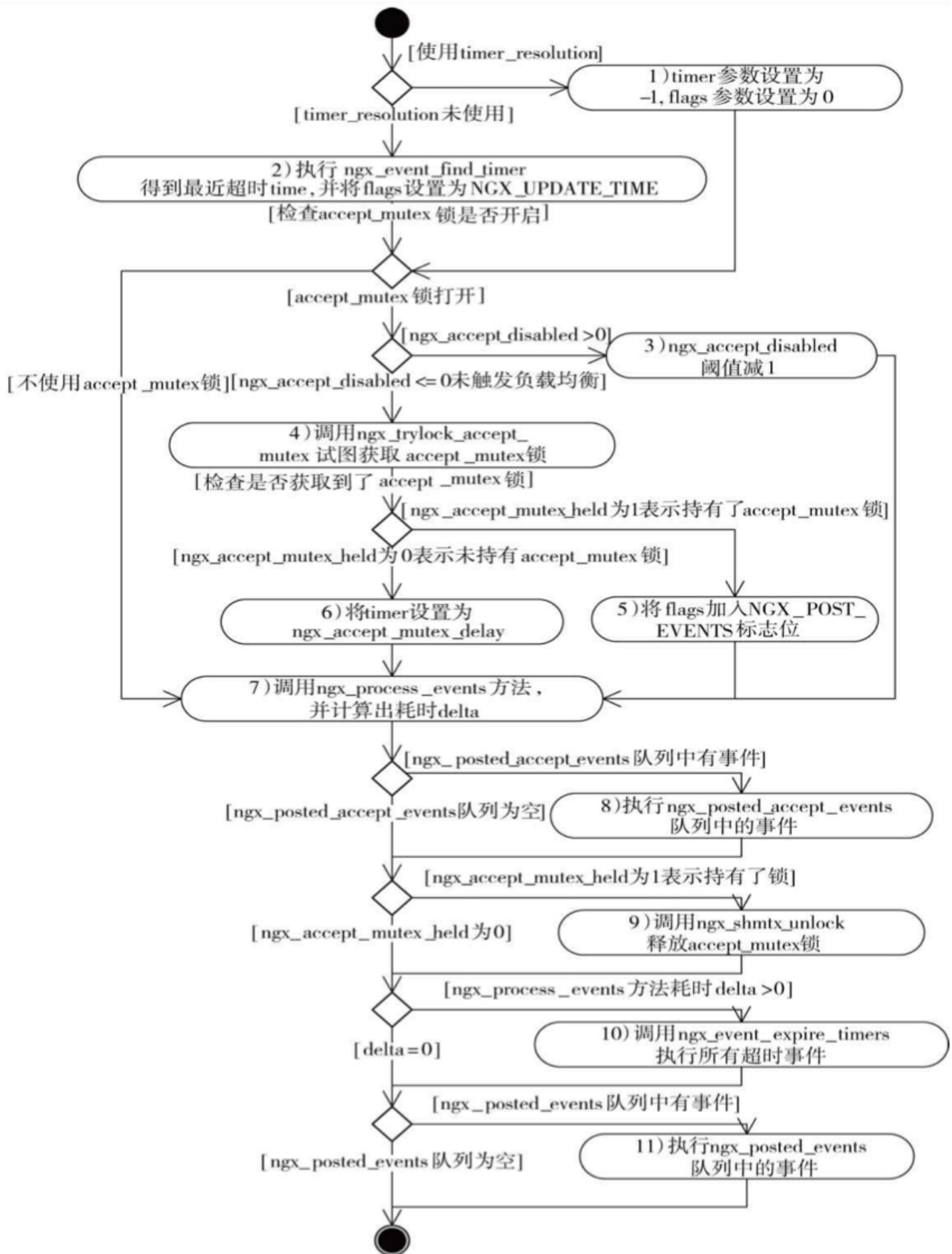
细节多久进行一次超时检测

红黑树没必要一直轮询去探测超时，而是等待红黑树中最左边的元素的超时时间后再去探测。

ngx_process_events_and_timers 整体流程讲解

在这个方法内我们需要了解几个点：

1. 如何实现负载均衡
2. 如何解决惊群事件
3. 如何进行连接处理



```
src/event/ngx_event.c
```

```
void
ngx_process_events_and_timers(ngx_cycle_t *cycle)
```

```

{
    ngx_uint_t  flags;
    ngx_msec_t  timer, delta;
    //如果开启了则意味着客户端返回数据在毫秒级，不进行超时事件的检查
    if (ngx_timer_resolution) {
        timer = NGX_TIMER_INFINITE;
        flags = 0;

    } else {
        timer = ngx_event_find_timer();
        flags = NGX_UPDATE_TIME;
    }

    /*
    ngx_use_accept_mutex变量代表是否使用accept互斥体
    默认是使用，可以通过accept_mutex off;指令关闭；
    accept_mutex 的作用就是避免惊群，同时实现负载均衡
    */
    if (ngx_use_accept_mutex) {

        /*
        ngx_accept_disabled变量在ngx_event_accept函数中计算。
        如果ngx_accept_disabled大于0，就表示该进程接受的链接过多，
        因此放弃一次争抢accept_mutex的机会，同时将自己减一。
        然后，继续处理已有连接上的事件。
        nginx就利用这一点实现了继承关于连接的基本负载均衡。
        */
        if (ngx_accept_disabled > 0) {
            ngx_accept_disabled--;

        } else {
            /*
            尝试锁accept_mutex，只有成功获取锁的进程，才会将listen套接字放到epoll
            中。

            因此，这就保证了只有一个进程拥有监听套接口，故所有进程阻塞在epoll_wait
            时，

            才不会惊群现象。
            */
            if (ngx_trylock_accept_mutex(cycle) == NGX_ERROR) {
                return;
            }

            if (ngx_accept_mutex_held) {
                /*
                如果进程获得了锁，将添加一个 NGX_POST_EVENTS 标志。
                这个标志的作用是将所有产生的事件放入一个队列中，等释放后，在慢慢来处理
                事件。

                因为，处理时间可能会很耗时，如果不先施放锁再处理的话，该进程就长时间霸
                占了锁，

                导致其他进程无法获取锁，这样accept的效率就低了。
                */

```



```

        flags |= NGX_POST_EVENTS;

    } else {
        /*
         * 没有获得所需进程，当然不需要NGX_POST_EVENTS标志。
         * 但需要设置延时多长时间，再去争抢锁。
         */
        if (timer == NGX_TIMER_INFINITE
            || timer > ngx_accept_mutex_delay)
        {
            timer = ngx_accept_mutex_delay;
        }
    }
}

```

```
delta = ngx_current_msec;
```

/*接下来，epoll要开始wait事件，
ngx_process_events的具体实现是对应到epoll模块中的ngx_epoll_process_events函

数

这里之后会详细讲解的哦

```

*/
(void) ngx_process_events(cycle, timer, flags);
//统计本次wait事件的耗时
delta = ngx_current_msec - delta;

```

```

ngx_log_debug1(NGX_LOG_DEBUG_EVENT, cycle->log, 0,
               "timer delta: %M", delta);

```

/*
ngx_posted_accept_events是一个事件队列，暂存epoll从监听套接口wait到的accept事

件。

前文提到的NGX_POST_EVENTS标志被使用后，会将所有的accept事件暂存到这个队列

```

*/
if (ngx_posted_accept_events) {
    ngx_event_process_posted(cycle, &ngx_posted_accept_events);
}
//所有accept事件处理完之后，如果持有锁的话，就释放掉。
if (ngx_accept_mutex_held) {
    ngx_shmtx_unlock(&ngx_accept_mutex);
}

```

```

/*
 * delta是之前统计的耗时，存在毫秒级的耗时，就对所有时间的timer进行检查，
 * 如果timeout 就从time rbtree中删除到期的timer，同时调用相应事件的handler函数处理
 */
if (delta) {
    ngx_event_expire_timers();
}

```

```
    ngx_log_debug1(NGX_LOG_DEBUG_EVENT, cycle->log, 0,
        "posted events %p", ngx_posted_events);

    /*
     * 处理普通事件(连接上获得的读写事件),
     * 因为每个事件都有自己的handler方法,
     */
    if (ngx_posted_events) {
        if (ngx_threaded) {
            ngx_wakeup_worker_thread(cycle);

        } else {
            ngx_event_process_posted(cycle, &ngx_posted_events);
        }
    }
}
```

负载均衡

可以看到ngx_accept_disabled > 0的时候worker进程并不会去竞争锁。而ngx_accept_disabled变量的初始化是在ngx_event_accept方法中的, 该方法我们在介绍事件驱动模块连接处理的时候会详细介绍, 先截取部分代码如下:

```
/*
 * ngx_accept_disabled 变量是负载均衡阈值, 表示进程是否超载;
 * 设置负载均衡阈值为每个进程最大连接数的八分之一减去空闲连接数;
 * 即当每个进程accept到的活动连接数超过最大连接数的7/8时,
 * ngx_accept_disabled 大于0, 表示该进程处于负载过重;
 */
ngx_accept_disabled = ngx_cycle->connection_n / 8
    - ngx_cycle->free_connection_n;
```

惊群问题

什么是惊群事件

惊群是指多个进程/线程在等待同一资源时, 每当资源可用, 所有的进程/线程都来竞争资源的现象。

Nginx采用的是多进程的模式。假设Linux系统是2.6版本以前, 当有一个客户端要连到Nginx服务器上, Nginx的N个进程都会去监听socket的accept的, 如果全部的N个进程都对这个客户端的socket连接进行了监听, 就会造成资源的竞争甚至数据的错乱。我们要保证的是, 一个链接在Nginx的一个进程上处理, 包括accept和read/write事件。

linux2.6 内核做了优化详细介绍: <https://blog.csdn.net/lyztyycode/article/details/78648798>

nginx是如何解决惊群的

- 通过ngx_trylock_accept_mutex争抢共享锁，其实就是利用mmap或shmget创建的共享内存中的变量，才可以处理accept事件。
- ngx_accept_mutex_held是拿到锁的一个标志，当拿到锁了，flags会被设置成NGX_POST_EVENTS，这个标志会在事件处理函数ngx_process_events中将所有事件（accept和read）放入对应的ngx_posted_accept_events和ngx_posted_events队列中进行延后处理。
- 当没有拿到锁，调用事件处理函数ngx_process_events的时候，可以明确都是read的事件，所以可以直接调用事件ev->handler方法回调处理。拿到锁的进程，接下来会优先处理ngx_posted_accept_events队列上的accept事件，处理函数：ngx_event_process_posted
- 处理完accept事件后，就将锁释放
- 接下来处理ngx_posted_events队列上的read事件，处理函数：ngx_event_process_posted

锁的获取

```
/**
 * 获取accept锁
 */
ngx_int_t ngx_trylock_accept_mutex(ngx_cycle_t *cycle) {
    /**
     * 拿到锁
     */
    if (ngx_shmtx_trylock(&ngx_accept_mutex)) {

        ngx_log_debug0(NGX_LOG_DEBUG_EVENT, cycle->log, 0,
            "accept mutex locked");

        /* 多次进来，判断是否已经拿到锁 */
        if (ngx_accept_mutex_held && ngx_accept_events == 0) {
            return NGX_OK;
        }

        /* 调用ngx_enable_accept_events，开启监听accept事件*/
        if (ngx_enable_accept_events(cycle) == NGX_ERROR) {
            ngx_shmtx_unlock(&ngx_accept_mutex);
            return NGX_ERROR;
        }

        ngx_accept_events = 0;
        ngx_accept_mutex_held = 1;

        return NGX_OK;
    }

    ngx_log_debug1(NGX_LOG_DEBUG_EVENT, cycle->log, 0,
        "accept mutex lock failed: %ui", ngx_accept_mutex_held);

    /**
     * 没有拿到锁，但是ngx_accept_mutex_held=1
     */
    if (ngx_accept_mutex_held) {
```

```

        /* 没有拿到锁，调用ngx_disable_accept_events，将accept事件删除 */
        if (ngx_disable_accept_events(cycle, 0) == NGX_ERROR) {
            return NGX_ERROR;
        }

        ngx_accept_mutex_held = 0;
    }

    return NGX_OK;
}

```

事件的添加和删除

```

/**
 * 开启accept事件的监听
 * 并将accept事件加入到event上
 */
static ngx_int_t ngx_enable_accept_events(ngx_cycle_t *cycle) {
    ngx_uint_t i;
    ngx_listening_t *ls;
    ngx_connection_t *c;

    ls = cycle->listening.elts;
    for (i = 0; i < cycle->listening.nelts; i++) {

        c = ls[i].connection;

        if (c == NULL || c->read->active) {
            continue;
        }

        if (ngx_add_event(c->read, NGX_READ_EVENT, 0) == NGX_ERROR) {
            return NGX_ERROR;
        }
    }

    return NGX_OK;
}

/**
 * 关闭accept事件的监听
 * 并将accept事件从event上删除
 */
static ngx_int_t ngx_disable_accept_events(ngx_cycle_t *cycle, ngx_uint_t
all) {
    ngx_uint_t i;
    ngx_listening_t *ls;
    ngx_connection_t *c;

```

```

ls = cycle->listening.elts;
for (i = 0; i < cycle->listening.nelts; i++) {

    c = ls[i].connection;

    /* 如果c->read->active, 则表示是活跃的连接, 已经被使用中 */
    if (c == NULL || !c->read->active) {
        continue;
    }

#ifdef NGX_HAVE_REUSEPORT

    /*
     * do not disable accept on worker's own sockets
     * when disabling accept events due to accept mutex
     */

    if (ls[i].reuseport && !all) {
        continue;
    }

#endif

    /* 删除事件 */
    if (ngx_del_event(c->read, NGX_READ_EVENT,
                     NGX_DISABLE_EVENT) == NGX_ERROR) {
        return NGX_ERROR;
    }
}

return NGX_OK;
}

```

事件驱动模块连接处理

在ngx_event_process_init方法中将每个connection的handler处理方法都初始化成ngx_event_accept方法。

1. 连接事件处理

```

/* 处理新连接事件 */
void
ngx_event_accept(ngx_event_t *ev)
{
    socklen_t      socklen;
    ngx_err_t      err;
    ngx_log_t      *log;
    ngx_uint_t      level;
    ngx_socket_t    s;
    ngx_event_t     *rev, *wev;

```

```
    ngx_listening_t    *ls;
    ngx_connection_t    *c, *lc;
    ngx_event_conf_t    *ecf;
    u_char              sa[NGX_SOCKADDRLEN];

    if (ev->timedout) {
        if (ngx_enable_accept_events((ngx_cycle_t *) ngx_cycle) !=
NGX_OK) {
            return;
        }

        ev->timedout = 0;
    }

    /* 获取ngx_event_core_module模块的配置项参数结构 */
    ecf = ngx_event_get_conf(ngx_cycle->conf_ctx,
ngx_event_core_module);

    if (ngx_event_flags & NGX_USE_RTSIG_EVENT) {
        ev->available = 1;
    } else if (!(ngx_event_flags & NGX_USE_KQUEUE_EVENT)) {
        ev->available = ecf->multi_accept;
    }

    lc = ev->data; /* 获取事件所对应的连接对象 */
    ls = lc->listening; /* 获取连接对象的监听端口数组 */
    ev->ready = 0; /* 设置事件的状态为未准备就绪 */

    ngx_log_debug2(NGX_LOG_DEBUG_EVENT, ev->log, 0,
        "accept on %V, ready: %d", &ls->addr_text, ev-
>available);

    do {
        socklen = NGX_SOCKADDRLEN;

        /* accept 建立一个新的连接 */
#ifdef NGX_HAVE_ACCEPT4
        if (use_accept4) {
            s = accept4(lc->fd, (struct sockaddr *) sa, &socklen,
                SOCK_NONBLOCK);
        } else {
            s = accept(lc->fd, (struct sockaddr *) sa, &socklen);
        }
#else
        s = accept(lc->fd, (struct sockaddr *) sa, &socklen);
#endif

        /* 连接建立错误时的相应处理 */
        if (s == (ngx_socket_t) -1) {
            err = ngx_socket_errno;
        }
    } while (s == (ngx_socket_t) -1);
```

```

        if (err == NGX_EAGAIN) {
            ngx_log_debug0(NGX_LOG_DEBUG_EVENT, ev->log, err,
                "accept() not ready");
            return;
        }

        level = NGX_LOG_ALERT;

        if (err == NGX_ECONNABORTED) {
            level = NGX_LOG_ERR;

        } else if (err == NGX_EMFILE || err == NGX_ENFILE) {
            level = NGX_LOG_CRIT;
        }

#ifdef NGX_HAVE_ACCEPT4
        ngx_log_error(level, ev->log, err,
            use_accept4 ? "accept4() failed" : "accept()
failed");

        if (use_accept4 && err == NGX_ENOSYS) {
            use_accept4 = 0;
            ngx_inherited_nonblocking = 0;
            continue;
        }
#else
        ngx_log_error(level, ev->log, err, "accept() failed");
#endif

        if (err == NGX_ECONNABORTED) {
            if (ngx_event_flags & NGX_USE_KQUEUE_EVENT) {
                ev->available--;
            }

            if (ev->available) {
                continue;
            }
        }

        if (err == NGX_EMFILE || err == NGX_ENFILE) {
            if (ngx_disable_accept_events((ngx_cycle_t *)
ngx_cycle)
                != NGX_OK)
            {
                return;
            }

            if (ngx_use_accept_mutex) {
                if (ngx_accept_mutex_held) {
                    ngx_shmtx_unlock(&ngx_accept_mutex);

```

```

        ngx_accept_mutex_held = 0;
    }

    ngx_accept_disabled = 1;

    } else {
        ngx_add_timer(ev, ecf->accept_mutex_delay);
    }
}

return;
}

#ifdef (NGX_STAT_STUB)
    (void) ngx_atomic_fetch_add(ngx_stat_accepted, 1);
#endif

/*
 * ngx_accept_disabled 变量是负载均衡阈值，表示进程是否超载；
 * 设置负载均衡阈值为每个进程最大连接数的八分之一减去空闲连接数；
 * 即当每个进程accept到的活动连接数超过最大连接数的7/8时，
 * ngx_accept_disabled 大于0，表示该进程处于负载过重；
 */
ngx_accept_disabled = ngx_cycle->connection_n / 8
                      - ngx_cycle->free_connection_n;

/* 从connections数组中获取一个connection连接来维护新的连接 */
c = ngx_get_connection(s, ev->log);

if (c == NULL) {
    if (ngx_close_socket(s) == -1) {
        ngx_log_error(NGX_LOG_ALERT, ev->log,
ngx_socket_errno,
                        ngx_close_socket_n " failed");
    }

    return;
}

#ifdef (NGX_STAT_STUB)
    (void) ngx_atomic_fetch_add(ngx_stat_active, 1);
#endif

/* 为新的连接创建一个连接池pool，直到关闭该连接时才释放该连接池pool */
c->pool = ngx_create_pool(ls->pool_size, ev->log);
if (c->pool == NULL) {
    ngx_close_accepted_connection(c);
    return;
}

c->sockaddr = ngx_palloc(c->pool, socklen);

```



```

        if (c->sockaddr == NULL) {
            ngx_close_accepted_connection(c);
            return;
        }

        ngx_memcpy(c->sockaddr, sa, socklen);

        log = ngx_palloc(c->pool, sizeof(ngx_log_t));
        if (log == NULL) {
            ngx_close_accepted_connection(c);
            return;
        }

        /* set a blocking mode for aio and non-blocking mode for
        others */

        /* 设置套接字的属性 */
        if (ngx_inherited_nonblocking) {
            if (ngx_event_flags & NGX_USE_AIO_EVENT) {
                if (ngx_blocking(s) == -1) {
                    ngx_log_error(NGX_LOG_ALERT, ev->log,
ngx_socket_errno,
                                ngx_blocking_n " failed");
                    ngx_close_accepted_connection(c);
                    return;
                }
            }

        } else {
            /* 使用epoll模型时, 套接字的属性为非阻塞模式 */
            if (!(ngx_event_flags &
(NGX_USE_AIO_EVENT|NGX_USE_RTSG_EVENT))) {
                if (ngx_nonblocking(s) == -1) {
                    ngx_log_error(NGX_LOG_ALERT, ev->log,
ngx_socket_errno,
                                ngx_nonblocking_n " failed");
                    ngx_close_accepted_connection(c);
                    return;
                }
            }
        }

        *log = ls->log;

        /* 初始化新连接 */
        c->recv = ngx_recv;
        c->send = ngx_send;
        c->recv_chain = ngx_recv_chain;
        c->send_chain = ngx_send_chain;

        c->log = log;

```

```

        c->pool->log = log;

        c->socklen = socklen;
        c->listening = ls;
        c->local_sockaddr = ls->sockaddr;
        c->local_socklen = ls->socklen;

        c->unexpected_eof = 1;

#ifdef NGX_HAVE_UNIX_DOMAIN
        if (c->sockaddr->sa_family == AF_UNIX) {
            c->tcp_nopush = NGX_TCP_NOPUSH_DISABLED;
            c->tcp_nodelay = NGX_TCP_NODELAY_DISABLED;
#ifdef NGX_SOLARIS
            /* Solaris's sendfilev() supports AF_NCA, AF_INET, and
AF_INET6 */
            c->sendfile = 0;
#endif
        }
#endif

        /* 获取新连接的读事件、写事件 */
        rev = c->read;
        wev = c->write;

        /* 写事件准备就绪 */
        wev->ready = 1;

        if (ngx_event_flags & (NGX_USE_AIO_EVENT|NGX_USE_RTSG_EVENT))
        {
            /* rtsig, aio, iocp */
            rev->ready = 1;
        }

        if (ev->deferred_accept) {
            rev->ready = 1;
#ifdef NGX_HAVE_KQUEUE
            rev->available = 1;
#endif
        }

        rev->log = log;
        wev->log = log;

        /*
        * TODO: MT: - ngx_atomic_fetch_add()
        *             or protection by critical section or light mutex
        *
        * TODO: MP: - allocated in a shared memory
        *             - ngx_atomic_fetch_add()
        *             or protection by critical section or light mutex

```

```

        */

        c->number = ngx_atomic_fetch_add(ngx_connection_counter, 1);

#if (NGX_STAT_STUB)
        (void) ngx_atomic_fetch_add(ngx_stat_handled, 1);
#endif

#if (NGX_THREADS)
        rev->lock = &c->lock;
        wev->lock = &c->lock;
        rev->own_lock = &c->lock;
        wev->own_lock = &c->lock;
#endif

        if (ls->addr_ntop) {
            c->addr_text.data = ngx_pnalloc(c->pool, ls->addr_text_max_len);
            if (c->addr_text.data == NULL) {
                ngx_close_accepted_connection(c);
                return;
            }

            c->addr_text.len = ngx_sock_ntop(c->sockaddr, c->socklen,
                                             c->addr_text.data,
                                             ls->addr_text_max_len, 0);

            if (c->addr_text.len == 0) {
                ngx_close_accepted_connection(c);
                return;
            }
        }

#if (NGX_DEBUG)
        {
            struct sockaddr_in    *sin;
            ngx_cidr_t            *cidr;
            ngx_uint_t            i;
#if (NGX_HAVE_INET6)
            struct sockaddr_in6   *sin6;
            ngx_uint_t            n;
#endif
#endif

            cidr = ecf->debug_connection.elts;
            for (i = 0; i < ecf->debug_connection.nelts; i++) {
                if (cidr[i].family != (ngx_uint_t) c->sockaddr->sa_family)
                    goto next;
            }

            switch (cidr[i].family) {

```

```

    #if (NGX_HAVE_INET6)
        case AF_INET6:
            sin6 = (struct sockaddr_in6 *) c->sockaddr;
            for (n = 0; n < 16; n++) {
                if ((sin6->sin6_addr.s6_addr[n]
                    & cidr[i].u.in6.mask.s6_addr[n])
                    != cidr[i].u.in6.addr.s6_addr[n])
                {
                    goto next;
                }
            }
            break;
    #endif

    #if (NGX_HAVE_UNIX_DOMAIN)
        case AF_UNIX:
            break;
    #endif

    default: /* AF_INET */
        sin = (struct sockaddr_in *) c->sockaddr;
        if ((sin->sin_addr.s_addr & cidr[i].u.in.mask)
            != cidr[i].u.in.addr)
        {
            goto next;
        }
        break;
    }

    log->log_level =
NGX_LOG_DEBUG_CONNECTION|NGX_LOG_DEBUG_ALL;
    break;

    next:
        continue;
    }

}

#endif

    ngx_log_debug3(NGX_LOG_DEBUG_EVENT, log, 0,
        "%uA accept: %V fd:%d", c->number, &c->addr_text,
s);

    /* 将新连接对应的读事件注册到事件监控机制中;
    * 注意: 若是epoll事件机制, 这里是不会执行,
    * 因为epoll事件机制会在调用新连接处理函数ls->handler(c)
    * (实际调用ngx_http_init_connection) 时, 才会把新连接对应的读事件注册
    到epoll事件机制中;
    */

```

```

        if (ngx_add_conn && (ngx_event_flags & NGX_USE_EPOLL_EVENT) ==
0) {
            if (ngx_add_conn(c) == NGX_ERROR) {
                ngx_close_accepted_connection(c);
                return;
            }

            log->data = NULL;
            log->handler = NULL;

            /*
             * 设置回调函数，完成新连接的最后初始化工作，
             * 由函数ngx_http_init_connection完成
             */
            ls->handler(c);

            /* 调整事件available标志位，该标志位为1表示Nginx一次尽可能多建立新连接
            */
            if (ngx_event_flags & NGX_USE_KQUEUE_EVENT) {
                ev->available--;
            }

        } while (ev->available);
    }

```

2. 连接读事件加入监听事件中

```

/* 将监听socket连接的读事件加入到监听事件中 */
static ngx_int_t
ngx_enable_accept_events(ngx_cycle_t *cycle)
{
    ngx_uint_t      i;
    ngx_listening_t *ls;
    ngx_connection_t *c;

    /* 获取监听数组的首地址 */
    ls = cycle->listening.elts;
    /* 遍历整个监听数组 */
    for (i = 0; i < cycle->listening.nelts; i++) {

        /* 获取当前监听socket所对应的连接 */
        c = ls[i].connection;

        /* 当前连接的读事件是否处于active活跃状态 */
        if (c->read->active) {
            /* 若是处于active状态，表示该连接的读事件已经在事件监控对象中 */
            continue;
        }
    }

```

```

/* 若当前连接没有加入到事件监控对象中，则将该链接注册到事件监控中 */
if (ngx_event_flags & NGX_USE_RTSIG_EVENT) {

    if (ngx_add_conn(c) == NGX_ERROR) {
        return NGX_ERROR;
    }

} else {
    /* 若当前连接的读事件不在事件监控对象中，则将其加入 */
    if (ngx_add_event(c->read, NGX_READ_EVENT, 0) ==
NGX_ERROR) {
        return NGX_ERROR;
    }
}

return NGX_OK;
}

```

3. 连接读事件从事件驱动模块中移除

```

/* 将监听连接的读事件从事件驱动模块中删除 */
static ngx_int_t
ngx_disable_accept_events(ngx_cycle_t *cycle)
{
    ngx_uint_t      i;
    ngx_listening_t *ls;
    ngx_connection_t *c;

    /* 获取监听接口 */
    ls = cycle->listening.elts;
    for (i = 0; i < cycle->listening.nelts; i++) {

        /* 获取监听接口对应的连接 */
        c = ls[i].connection;

        if (!c->read->active) {
            continue;
        }

        /* 从事件驱动模块中移除连接 */
        if (ngx_event_flags & NGX_USE_RTSIG_EVENT) {
            if (ngx_del_conn(c, NGX_DISABLE_EVENT) == NGX_ERROR) {
                return NGX_ERROR;
            }
        }

    } else {
        /* 从事件驱动模块移除连接的读事件 */
    }
}

```

```
        if (ngx_del_event(c->read, NGX_READ_EVENT,  
NGX_DISABLE_EVENT)  
            == NGX_ERROR)  
        {  
            return NGX_ERROR;  
        }  
    }  
}  
  
    return NGX_OK;  
}
```

TCP背景知识

TCP连接建立

TCP发送数据

TCP接受数据